Università degli Studi di Modena e Reggio Emilia

Department of Physics, Informatics and Mathematics

Bachelor's Degree in Computer Science

Assignment Two: Data-Flow Analysis

Compilers – Part II

Authors:
Alessio Yang
Christian Zanetti

Professor:
Andrea Marongiu

Academic Year 2025/2026

Contents

1 Introduction

Data-Flow Analysis (DFA) is a family of techniques used in a compiler's middle-end to statically gather information about the possible set of values calculated at various points in a program along all execution paths. By examining the Control Flow Graph (CFG) of a program, DFA algorithms evaluate the semantics of each node to compute the data state at its entry and exit points. Gathering this information allows the compiler to apply optimizations. The core idea is to set up data-flow equations for each node in the CFG and solve them iteratively until a fixed point is reached. A fixed point is considered reached when the results of two successive iterations coincide. A typical data-flow analysis is defined by a domain of values, a traversal direction (forward or backward), a set of transfer functions, meet/join operations, a boundary condition (which dictates the data-flow state at the `ENTRY` or `EXIT` node of the CFG), and initial interior points (the starting assumption assigned to all other nodes — typically the empty set $\emptyset$ or the universal set $\mathcal{U}$ — before the iterative solving begins).

Data-flow analysis serves as the foundation for a wide variety of program optimizations and analyses. The three DFA applications we will cover are as follows:

- Very Busy Expressions: An expression is considered *very busy* at a specific program point p if, along every possible execution path from p to the `EXIT` node, the expression is guaranteed to be evaluated before any of its operands are redefined.
- Dominator Analysis: A node d is said to *dominate* a node n if every possible path from the `ENTRY` node to n must pass through d. Every node dominates itself, and the `ENTRY` node dominates all other nodes.
- Constant Propagation: It determines whether a specific variable is guaranteed to hold a constant value at a given point in the program, regardless of the execution path taken to reach that point.

2 Very Busy Expressions

Very Busy Expressions can be formalized as a backward data-flow analysis. In this framework, the compiler tracks sets of expressions that are guaranteed to be evaluated before any of their operands are redefined along all possible future execution paths.

- Direction: Backward. An expression being busy at a point p depends on what happens after p, along all future execution paths. Consequently, to determine the exit state OUT of a given block, the framework uses information from the future, specifically the entry states IN of its successor nodes.
- Domain: Powerset of all expressions $\mathcal{P}(\mathcal{E})$, where $\mathcal{E}$ is the set of all expressions appearing in the program.
- Meet Operation: Set intersection $\cap$. When multiple execution paths diverge from a block B, an expression is considered very busy only if it is guaranteed to be evaluated along every outgoing path, and before any of its operands are redefined.

$$OUT(B) = \bigcap_{S \in SUCC(B)} IN(S)$$

- Boundary Condition: At the exit point of the program, no further instructions are executed, meaning no expressions can be evaluated subsequently. Therefore, no expression can be considered very busy at this point, and the boundary is initialized to the empty set.

$$IN(\text{EXIT}) = \emptyset$$

- Initial Interior Points: To ensure the intersection operator correctly converges to the fixed point, all internal blocks are initialized to the universal set $\mathcal{U}$, representing the assumption that all expressions are initially very busy. This prevents the intersection operator from discarding expressions (i.e., information) that should instead be propagated.
  The $\emptyset$ initialization is a particularly illustrative counterexample: since set intersection is defined as $A \cap \emptyset = \emptyset$ for any set A, initializing any block to $\emptyset$ would cause the meet operation to immediately collapse the entire data-flow state to $\emptyset$, destroying all information that the backward propagation was meant to carry.

$$IN(B) = \mathcal{U} \quad \forall B \neq \text{EXIT}$$

Before introducing the transfer function, it is worth defining the two auxiliary sets Gen_B and $Kill_B$, which capture the semantic effect of the instructions within block B on the set of very busy expressions:

- Gen_B (Generated expressions): This set represents the new data-flow information created by executing the block. Therefore, in this analysis it contains the expressions that are evaluated in B before any of their operands are redefined. We can

call these upward-exposed expressions. Any expression in Gen_B is guaranteed to be busy at the entry of B, regardless of the state at its exit.

- $Kill_B$ (Killed expressions): This set contains the expressions that are invalidated by block B. Specifically, an expression $E(x_1, \ldots, x_n)$ is killed in B if at least one of its operands x_i is redefined in B before E is evaluated. Any such expression can no longer be considered very busy at the entry of B, even if it was busy at its exit.

The transfer function for a block B propagates busy expressions backward through the block in the following way:

$$IN(B) = f(OUT(B)) = Gen_B \cup (OUT(B) \setminus Kill_B)$$

This corresponds to the idea that the set of very busy expressions at the entry of B ($IN(B)$) consists of all expressions that are locally upward-exposed (Gen_B), plus any expressions that are busy at the exit of B and have not been invalidated by a redefinition of their operands within B ($OUT(B) \setminus Kill_B$).

3 Dominator Analysis

Dominator Analysis can be formalized as a forward data-flow analysis. In this framework, the compiler tracks sets of nodes (basic blocks) that are guaranteed to have been visited along every possible path from the `ENTRY` node to the current block.

- Direction: Forward. Whether a node d dominates a node n depends entirely on the execution paths taken before reaching n. Consequently, to determine the entry state IN of a given block, the framework uses information from the past, specifically the exit states OUT of its predecessor nodes.
- Domain: Powerset of all basic blocks $\mathcal{P}(\mathcal{B})$, where $\mathcal{B}$ is the set of all basic blocks in the CFG. Each element in the domain is a set of basic blocks, representing the dominators of a given node.
- Meet Operation: Set intersection $\cap$. When multiple execution paths converge at a block B, a node d is considered a dominator of B only if it is present in *all* the incoming paths. Therefore, the dominators of B are exactly the intersection of the dominator sets of its predecessors.

$$IN(B) = \bigcap_{P \in PRED(B)} OUT(P)$$

- Boundary Condition: The `ENTRY` node has no predecessors. By definition, the only node that can dominate the `ENTRY` node is the `ENTRY` node itself.

$$OUT(\text{ENTRY}) = \{\text{ENTRY}\}$$

- Initial Interior Points: To ensure convergence, all internal blocks are initialized to the universal set $\mathcal{U}$ (which, in this context, is the set of all basic blocks $\mathcal{B}$). As we explained before in the Very Busy Expressions framework, whenever the meet operator is set intersection, initializing interior points to $\mathcal{U}$ is necessary to avoid the discard of information that should instead be propagated.

$$OUT(B) = \mathcal{U} \quad \forall B \neq \text{ENTRY}$$

To define the transfer function, we must evaluate how the execution of a block B affects the dominance data-flow state through its Gen_B and $Kill_B$ sets:

- Gen_B (Generated dominators): By definition, every block always dominates itself. Therefore, reaching and executing block B inherently generates the fact that B is a dominator for any subsequent node along that path. Thus, $Gen_B = \{B\}$.
- $Kill_B$ (Killed dominators): The execution of block B does not invalidate or alter the sequence of nodes visited prior to reaching it. No previously established dominators are destroyed, meaning $Kill_B = \emptyset$. It is worth noting that an empty kill set

does not imply that dominance information propagates unchecked throughout the CFG. Instead, the necessary "filtering" of the data-flow state is naturally enforced by the intersection meet operator, which discards any node that is not present along all converging execution paths.

Given the standard transfer function for a forward analysis:

$$OUT(B) = f(IN(B)) = Gen_B \cup (IN(B) \setminus Kill_B)$$

By substituting the specific sets for the Dominator Analysis ($Gen_B = \{B\}$ and $Kill_B = \emptyset$), the equation drastically simplifies to:

$$OUT(B) = \{B\} \cup IN(B)$$

Intuitively, this simplified function dictates that the set of dominators at the exit of block B ($OUT(B)$) simply consists of all the dominators inherited from its predecessors ($IN(B)$), plus the block B itself.

4 Constant Propagation

Constant Propagation can be formalized as a forward data-flow analysis. In this framework, the compiler tracks sets of variables whose values are statically guaranteed to be constant at specific program points.

- Direction: Forward. The assignment of a value to a variable at one point in the program only affects the state of the instructions that will be executed subsequently. Consequently, to determine the entry state IN of a given block, the framework uses information from the past, specifically the exit states OUT of its predecessor nodes.
- Domain: Powerset of all possible variable-constant pairs $\mathcal{P}(\mathcal{V} \times \mathcal{C})$. Each element in the domain is a set of tuples $\langle v, c \rangle$, indicating that the variable v is guaranteed to hold the constant value c. This definition implies that, compared to the previous analyses, the domain is mathematically infinite. Later, we must find a way to address this complexity during the actual implementation.
- Meet Operation: Set intersection $\cap$. When multiple execution paths converge at a block B, a variable is considered constant only if it holds the exact same constant value along all incoming paths.

$$IN(B) = \bigcap_{P \in PRED(B)} OUT(P)$$

- Boundary Condition: At the entry point of the program, no variables have statically known constant values.

$$OUT(\text{ENTRY}) = \emptyset$$

- Initial Interior Points: To ensure the intersection operator correctly converges to the fixed point, all internal blocks are initialized to the universal set $\mathcal{U} = \mathcal{V} \times \mathcal{C}$ (the set containing all possible variable-constant pairs).

$$OUT(B) = \mathcal{U} \quad \forall B \neq \text{ENTRY}$$

To fully understand the transfer function, it is essential to define the roles of the Gen_B and $Kill_B$ sets, which model the semantic impact of the instructions within block B:

- Gen_B (Generated pairs): As we previously discussed, this set represents the new data-flow information created by executing the block. Any pair generated here is guaranteed to hold true at the block's exit. For instance, an assignment like `x = 5` generates the pair $\langle x, 5 \rangle$.

- $Kill_B$ (Killed pairs): This set contains the data-flow information from the input state ($IN(B)$) that is invalidated, overridden, or destroyed by the block's instructions. For example, any new assignment to a variable `x` *kills* all previous assumptions about `x`'s value. Intuitively, since a kill operation must invalidate every possible constant associated with a redefined variable, the $Kill_B$ set is theoretically infinite. We will discuss how to manage this challenge later using a symbolic approach.

The standard transfer function for a block B relies on the sets of generated and killed pairs:

$$OUT(B) = f(IN(B)) = Gen_B \cup (IN(B) \setminus Kill_B)$$

Intuitively, the equation dictates that the output state ($OUT(B)$) consists of all newly generated pairs (Gen_B), plus any prior pairs from the input that managed to survive without being invalidated by the block ($IN(B) \setminus Kill_B$).

However, the way Gen_B and $Kill_B$ are defined determines the actual optimization power of the compiler.

4.1 Static Gen-Kill Sets (Direct Constant Propagation Only)

In classical data-flow analysis, Gen_B and $Kill_B$ are computed *statically* by inspecting the code once before the iterative algorithm begins.

- If a block contains an explicit assignment like `x = 5`, the framework statically generates $\langle x, 5 \rangle$.
- Limitation: If a block contains an arithmetic expression like `z = x + y` and `x` and `y` turn out to be constants during the subsequent iterative phase, static inspection yields $Gen_B = \emptyset$ because there are no explicit numeric constants in the text. Consequently, the framework will only propagate explicitly assigned constants but will completely fail to evaluate mathematical expressions.

Note on Domain Finiteness and Code Representation Despite these limitations, this static approach remains representable within the classical bit-vector framework because the domain of data-flow facts is strictly finite. Since we only consider *explicitly assigned* constants (literals), the total number of pairs ⟨variable, value⟩ is bounded by the product of the number of unique variables and the number of unique literals appearing in the source code. While this restriction prevents the evaluation of derived constants (e.g., `z = x + y`) or copies (e.g., `y = x`), it ensures that the bit-vectors have a finite length, allowing the iterative algorithm to reach a fixed point using standard bitwise intersections. This confirms that a simplified version of Constant Propagation is indeed a DFA, even though it has significantly reduced analytical power compared to a more comprehensive implementation.
It's worth noting that these limitations can be addressed by adopting a Static Single Assignment (SSA) representation. By ensuring each variable is defined exactly once, the

challenge of tracking derived constants (e.g., `y = x`) would be simplified, as the unique naming convention of SSA could allow the analysis to directly link the constant value of a source variable to all its subsequent uses. This suggests that the constraints of the classical DFA observed here might be a consequence of the code representation rather than a fundamental limit of data-flow techniques themselves.
Nevertheless, the following sections explore attempts to provide alternative solutions to better describe the problem of Constant Folding, by keeping the same representation.

4.2 Dynamic Gen-Kill Sets (True Constant Folding)

To achieve true *Constant Folding* (evaluating expressions at compile time), the Gen_B and $Kill_B$ sets cannot be fixed. They must be evaluated *dynamically* during each iteration as functions of the current input state $IN(B)$. The transfer function is modified as follows:

$$OUT(B) = Gen_B(IN(B)) \cup (IN(B) \setminus Kill_B(IN(B)))$$

When the analyzer processes an instruction like `z = x + y`:

- Dynamic Generation $Gen_B(IN(B))$: The analyzer inspects $IN(B)$. If $IN(B)$ currently contains $\langle x, c_1 \rangle$ and $\langle y, c_2 \rangle$, it evaluates the expression and dynamically generates the new constant pair $\langle z, c_1 + c_2 \rangle$.
- Dynamic Killing $Kill_B(IN(B))$: Any previous pair associated with z in $IN(B)$ is invalidated and removed, as the variable z has been redefined.

4.3 Symbolic representation of Universal Set and Kill Set

While initializing blocks with the universal set $\mathcal{U}$ guarantees the convergence of the algorithm, from an implementation perspective, we must quantify the actual size of $\mathcal{U}$. Given a 32-bit unsigned integer variable `x`, to represent the universal set for `x`, we would need to allocate over 4 billion pairs in memory:

$$\{\langle x, 0 \rangle, \langle x, 1 \rangle, \ldots, \langle x, 2^{32} - 1 \rangle\}$$

In a real-world program, the size of $\mathcal{U}$ would exhaust the computer's memory in fractions of a second. Therefore, we cannot represent the universal set *extensively*, but we can use a *symbolic* representation that captures the same logical meaning ("potentially any constant").

By representing $\mathcal{U}$ symbolically, we need to extend the definition of the intersection operator ($\cap$) with the following rules:

- **Symbolic Intersection**: $\mathcal{U} \cap \{\langle v, c \rangle\} = \{\langle v, c \rangle\}$
- **Symbolic Identity**: $\mathcal{U} \cap \mathcal{U} = \mathcal{U}$

Just as the universal set $\mathcal{U}$ requires a symbolic representation to avoid memory exhaustion, the $Kill_B$ sets must be handled similarly. When an instruction redefines a variable v, the framework must invalidate any previous constant associated with it. We introduce a symbol $\mathcal{C}$ which represents the set of all constants.

Thus, killing a variable v is denoted as generating the symbolic pair $\langle v, \mathcal{C} \rangle$. The set difference operator ($\setminus$) is then updated to interpret the symbol $\mathcal{C}$:

- **Symbolic Match**: $\{\langle v, c \rangle\} \setminus \{\langle v, \mathcal{C} \rangle\} = \emptyset$
- **Symbolic Mismatch**: $\{\langle x, c \rangle\} \setminus \{\langle v, \mathcal{C} \rangle\} = \{\langle x, c \rangle\} \quad \text{for } x \neq v$

In this way, we resolve the memory consumption issues caused by infinite sets. By representing them symbolically, we achieve a negligible memory footprint while preserving the integrity of the data-flow framework through the extension of the intersection and set difference operators.

Algorithm 1 Constant Propagation

Require: $CFG(N, E, ENTRY, EXIT)$
Ensure: $OUT[b] \quad \forall b \in N$
1: $OUT[ENTRY] \leftarrow \emptyset$
2: **for all** $b \in N \setminus \{ENTRY\}$ **do**
3: $\quad OUT[b] \leftarrow \mathcal{U}$ ▷ Symbolic Universal Set
4: **end for**
5: $set \leftarrow N \setminus \{ENTRY\}$
6: **while** $set \neq \emptyset$ **do**
7: $\quad b \leftarrow set.pop()$
8: $\quad IN[b] \leftarrow \bigcap_{p|(p,b)\in E} OUT[p]$ ▷ Meet Operation
9: $\quad old\_OUT \leftarrow OUT[b]$
10: $\quad Gen_b, Kill_b \leftarrow$ COMPUTE_DYNAMIC_GEN_KILL(b)
11: $\quad OUT[b] \leftarrow Gen_b \cup (IN[b] \setminus Kill_b)$
12: $\quad$ **if** $OUT[b] \neq old\_OUT$ **then** ▷ Convergence Check
13: $\quad\quad$ **for all** $(b, s) \in E$ **do**
14: $\quad\quad\quad$ **if** $s \notin set$ **then**
15: $\quad\quad\quad\quad set.add(s)$
16: $\quad\quad\quad$ **end if**
17: $\quad\quad$ **end for**
18: $\quad$ **end if**
19: **end while**

Algorithm 2 Dynamic update of sets Gen and Kill

Require: A basic block B
Ensure: Dynamically updated Gen_B and $Kill_B$ sets for B

function COMPUTE_DYNAMIC_GEN_KILL(B)
    $Gen_B \leftarrow \emptyset$
    $Kill_B \leftarrow \emptyset$
    $state$[1]$\leftarrow IN[B]$
    for all instruction $I \in B$ **do**
        if I is of form "$v = c$" **then** ▷ Explicit Constant
            $Kill_B \leftarrow Kill_B \cup \{\langle v, \mathcal{C}\rangle\}$ ▷ Symbolic Kill
            $Gen_B \leftarrow (Gen_B \setminus \{\langle v, \mathcal{C}\rangle\}) \cup \{\langle v, c\rangle\}$
            $state \leftarrow (state \setminus \{\langle v, \mathcal{C}\rangle\}) \cup \{\langle v, c\rangle\}$
        else if I is of form "$z = x$ op y" **then**
            if $\langle x, c_1\rangle \in state$ **and** $\langle y, c_2\rangle \in state$ **then**
                $c_{new} \leftarrow c_1$ op c_2 ▷ Constant Folding
                $Kill_B \leftarrow Kill_B \cup \{\langle z, \mathcal{C}\rangle\}$
                $Gen_B \leftarrow (Gen_B \setminus \{\langle z, \mathcal{C}\rangle\}) \cup \{\langle z, c_{new}\rangle\}$
                $state \leftarrow (state \setminus \{\langle z, \mathcal{C}\rangle\}) \cup \{\langle z, c_{new}\rangle\}$
            else
                $Kill_B \leftarrow Kill_B \cup \{\langle z, \mathcal{C}\rangle\}$ ▷ z is no longer constant
                $Gen_B \leftarrow Gen_B \setminus \{\langle z, \mathcal{C}\rangle\}$
                $state \leftarrow state \setminus \{\langle z, \mathcal{C}\rangle\}$
            end if
        end if
    end for
    return $Gen_B, Kill_B$
end function

[1] *state* represents the set of live constant variables in the current block

5 Framework Summary

	Very Busy Expressions
Direction	Backward $IN(B) = f_B(OUT(B))$ $OUT(B) = \bigwedge_{S \in SUCC(B)} IN(S)$[2]
Domain	$\mathcal{P}(\mathcal{E}) = \mathcal{P}(\{E \mid E \in \mathcal{E}\})$
Meet Operator	Intersection $\cap$
Boundary Condition	$IN(EXIT) = \emptyset$
Initial Interior Points	$IN(B) = \mathcal{U}$
Transfer Function	$IN(B) = f_B(OUT(B)) = Gen_B \cup (OUT(B) \setminus Kill_B)$

Table 1: Formal definition of the Very Busy Expressions framework.

	Dominator Analysis
Direction	Forward $OUT(B) = f_B(IN(B))$ $IN(B) = \bigwedge_{P \in PRED(B)} OUT(P)$
Domain	$\mathcal{P}(\mathcal{B}) = \mathcal{P}(\{B \mid B \in \mathcal{B}\})$
Meet Operator	Intersection $\cap$
Boundary Condition	$OUT(ENTRY) = ENTRY$
Initial Interior Points	$OUT(B) = \mathcal{U}$
Transfer Function	$OUT(B) = f_B(IN(B)) = Gen_B \cup (IN(B) \setminus Kill_B)$
Simplified Function	$OUT(B) = f_B(IN(B)) = \{B\} \cup IN(B)$

Table 2: Formal definition of the Dominator Analysis framework.

[2] $\bigwedge$ denotes the general meet operator.

	Constant Propagation
Direction	Forward $OUT(B) = f_B(IN(B))$ $IN(B) = \bigwedge_{P \in PRED(B)} OUT(P)$
Domain	$\mathcal{P}(\mathcal{V} \times \mathcal{C}) = \mathcal{P}(\{\langle v, c \rangle \mid v \in \mathcal{V}, c \in \mathcal{C}\})$
Meet Operator	Intersection $\cap$
Boundary Condition	$OUT(ENTRY) = \emptyset$
Initial Interior Points	$OUT(B) = \mathcal{U}$
Transfer Function	$OUT(B) = Gen_B(IN(B)) \cup (IN(B) \setminus Kill_B(IN(B)))$

Table 3: Formal definition of the Constant Propagation framework.